

GPUMODE Lecture Study Notes: Lecture 4 Compute and Memory Basics

Contents

1 Core Problem the Lecture Addresses

This lecture explains the two most important low-level ingredients behind fast GPU programs:

1. **How GPU computation is organized:** CUDA threads are grouped into warps, thread blocks, and grids, but the hardware actually executes them on streaming multiprocessors, abbreviated as SMs.
2. **How GPU memory traffic limits performance:** many kernels are not limited by the number of arithmetic operations they perform, but by how often they read from and write to global memory.

The main technical problem is:

A GPU is fast only when many SMs have enough active warps to execute, those warps avoid unnecessary divergence, and the kernel performs enough useful computation per byte moved from global memory.

The lecture studies chapters four and five of *Programming Massively Parallel Processors*. The focus is not merely how to launch CUDA kernels, but how the launched grid maps onto actual GPU hardware.

A CUDA programmer writes code in terms of:

- grids,
- thread blocks,
- threads,
- `threadIdx`,
- `blockIdx`,
- global memory pointers.

The GPU executes the code in terms of:

- SMs,
- warp schedulers,
- warps of 32 threads,
- register files,
- shared memory,
- L1 cache,
- L2 cache,
- global memory bandwidth,
- instruction issue and latency hiding.

The core message is that GPU performance is governed by both **compute occupancy** and **memory locality**. Good kernels keep the hardware busy while reducing expensive traffic to global memory.

2 Required Background

2.1 CUDA Execution Hierarchy

CUDA exposes a hierarchy:

grid = collection of thread blocks, (1)

thread block = collection of threads that can cooperate, (2)

warp = hardware execution group of 32 threads on NVIDIA GPUs. (3)

A kernel launch specifies:

```
kernel<<<grid_dim, block_dim>>>(arguments);
```

Here:

- `grid_dim` describes how many thread blocks are launched.
- `block_dim` describes how many threads are in each block.
- every thread block is assigned to one SM.
- a single SM may host multiple thread blocks at the same time if resources allow.
- threads in the same block can synchronize with `__syncthreads()`.
- different blocks are independent and cannot safely synchronize with each other inside a normal CUDA kernel.

2.2 Streaming Multiprocessors

A **streaming multiprocessor** is the main execution unit of an NVIDIA GPU. A modern GPU has many SMs. For example, the lecture uses an RTX 3090 as a concrete example, with approximately 82 SMs.

The SM contains:

- warp schedulers,
- dispatch units,
- FP32 arithmetic units,
- INT32 arithmetic units,
- tensor cores,
- a register file,
- L1 cache,
- shared memory.

The SM is not like a CPU core. A CPU core spends substantial hardware area on branch prediction, instruction decoding, speculative execution, and out-of-order scheduling. A GPU SM instead spends a much larger fraction of hardware area on arithmetic units and relies on massive thread-level parallelism to hide latency.

2.3 Warps

On NVIDIA GPUs, a warp contains 32 threads:

$$\text{warp size} = 32. \tag{4}$$

The warp is the unit at which many instructions are scheduled. Ideally, all 32 threads in a warp execute the same instruction at the same time on different data. This is often called SIMT, meaning single instruction, multiple threads.

For AMD GPUs, the analogous concept is often called a **wavefront**. AMD wavefronts are commonly 64 lanes, although some compiler modes can use 32-lane wavefronts.

2.4 Memory Hierarchy

The CUDA memory hierarchy includes:

- **registers**: fastest per-thread storage,
- **local memory**: per-thread memory, usually backed by global memory when registers spill,
- **shared memory**: fast per-block programmer-managed memory,
- **L1 cache**: per-SM cache,
- **L2 cache**: shared across SMs,
- **global memory**: large high-bandwidth device memory,
- **constant memory**: read-only memory often used for kernel parameters and constants.

The crucial performance fact is:

Global memory is large but relatively slow. Shared memory and registers are much faster but much smaller. Fast kernels reuse data in registers and shared memory instead of repeatedly loading it from global memory.

3 Main Concepts

3.1 CPU Core Versus GPU SM

A CPU core is optimized for latency and complex single-thread performance. It includes:

- instruction fetch,
- instruction decode,
- branch prediction,
- out-of-order execution,
- speculative execution,
- large caches,

- a smaller number of arithmetic units.

A GPU SM is optimized for throughput. It contains many arithmetic units and executes many lightweight threads. Instead of minimizing the latency of one thread, it hides latency by switching to other ready warps.

If one warp waits for memory, the SM can issue instructions from another warp whose operands are ready.

$$\text{GPU throughput} \approx \text{useful work per warp} \times \text{number of ready warps.} \quad (5)$$

The GPU therefore needs many active warps to remain efficient.

3.2 Thread Blocks Are Assigned to SMs

When a CUDA kernel launches, CUDA assigns thread blocks to SMs. The programmer does not usually control exactly which block goes to which SM.

Important consequences:

- blocks must be independent,
- block execution order is not guaranteed,
- blocks may execute concurrently or sequentially depending on available SM resources,
- one SM may host multiple blocks simultaneously,
- threads within one block can cooperate through shared memory.

If a GPU has S SMs and the kernel launches B blocks, then good SM-level parallelism usually requires:

$$B \gg S. \quad (6)$$

For a GPU with 82 SMs, launching only a small number of blocks leaves many SMs idle.

3.3 Maximum Resident Threads per SM

The lecture gives a concrete example using an RTX 3090-like GPU, where the maximum number of resident threads per SM is approximately:

$$T_{\text{max,SM}} = 1536. \quad (7)$$

If each block has T_{block} threads, then the maximum number of blocks that can fit by thread count alone is:

$$B_{\text{per SM}} = \left\lfloor \frac{T_{\text{max,SM}}}{T_{\text{block}}} \right\rfloor. \quad (8)$$

Examples:

$$T_{\text{block}} = 256 \qquad B_{\text{per SM}} = 6, \quad (9)$$

$$T_{\text{block}} = 512 \qquad B_{\text{per SM}} = 3, \quad (10)$$

$$T_{\text{block}} = 1024 \qquad B_{\text{per SM}} = 1. \quad (11)$$

A block size of 1024 may look large, but it can reduce flexibility because only one such block may fit on an SM. Smaller powers of two, such as 128, 256, or 512, often make it easier to maintain high occupancy, subject to register and shared-memory constraints.

3.4 Occupancy

Occupancy measures how many warps are active on an SM relative to the maximum possible number.

A simplified definition is:

$$\text{occupancy} = \frac{\text{active warps per SM}}{\text{maximum warps per SM}}. \quad (12)$$

High occupancy helps hide latency. If one warp stalls on memory, another ready warp can execute.

However, occupancy is not the same as performance. A kernel can have high occupancy but still be slow if:

- memory accesses are uncoalesced,
- arithmetic intensity is low,
- too many global memory operations are performed,
- the kernel uses slow operations such as FP64 on consumer GPUs,
- the kernel suffers heavy warp divergence,
- instruction mix or dependency chains limit throughput.

3.5 Registers and Register Pressure

Each SM has a finite register file. The lecture mentions a typical value:

$$R_{\text{SM}} = 65536 \quad (13)$$

32-bit registers per SM.

If each thread uses R_{thread} registers, then the number of threads that can be resident due to registers alone is bounded by:

$$T_{\text{resident}} \leq \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}}} \right\rfloor. \quad (14)$$

Thus, even if the hardware allows 1536 resident threads, high register usage can reduce the number of resident threads.

For example:

$$R_{\text{thread}} = 32 \quad T_{\text{resident}} \leq 2048, \quad (15)$$

$$R_{\text{thread}} = 64 \quad T_{\text{resident}} \leq 1024, \quad (16)$$

$$R_{\text{thread}} = 128 \quad T_{\text{resident}} \leq 512. \quad (17)$$

The last case can severely reduce occupancy.

If the compiler cannot keep all local variables in registers, it may spill them to local memory. Despite the name, **local memory** is usually backed by global memory and is far slower than registers.

Register pressure is a trade-off. More registers per thread may allow more computation per thread, but too many registers reduce occupancy and can cause spills.

3.6 Shared Memory and L1 Cache

On many NVIDIA GPUs, shared memory and L1 cache share a physical on-chip memory pool. The lecture mentions a combined pool of approximately 128 KB per SM on the referenced GPU generation.

This memory can be split between:

- programmer-managed shared memory,
- automatic L1 cache.

Shared memory is explicitly allocated inside a kernel:

```
__shared__ float tile[16][16];
```

Shared memory is:

- visible to all threads in the same block,
- much faster than global memory,
- useful for tiling,
- limited in size,
- a constraint on how many blocks can reside on an SM.

If each block uses $M_{\text{shared,block}}$ bytes of shared memory, then the shared-memory resource bound is:

$$B_{\text{per SM}} \leq \left\lfloor \frac{M_{\text{shared,SM}}}{M_{\text{shared,block}}} \right\rfloor. \quad (18)$$

3.7 FP64 and INT64 Can Be Slow on Consumer GPUs

The lecture emphasizes that consumer GPUs have much lower FP64 throughput than FP32 throughput. For example, a consumer GPU may have an FP64 rate around:

$$\text{FP64 throughput} \approx \frac{1}{64} \text{FP32 throughput}. \quad (19)$$

This means accidental use of 64-bit floating point constants can slow kernels dramatically. For example, writing:

```
float y = x * 0.5;
```

may involve a double-precision literal, while writing:

```
float y = x * 0.5f;
```

uses a single-precision literal.

The same caution applies to 64-bit integer indexing. Using `int64_t` for indexing inside a kernel can be slower than using 32-bit integer indexing when the tensor sizes permit 32-bit indices.

3.8 Warp Divergence

Warp divergence occurs when threads in the same warp take different control-flow paths.

Example:

```
if (threadIdx.x < 16) {  
    y = a + b;  
} else {  
    y = a * b;  
}
```

Within one warp, threads 0 through 15 take one branch, and threads 16 through 31 take the other. On traditional SIMT execution, the GPU executes one path while masking off the inactive threads, then executes the other path while masking off the opposite threads.

If a warp of 32 threads splits into two groups of 16 threads, then each branch uses only half the lanes.

A simplified efficiency model is:

$$\eta_{\text{branch}} = \frac{\text{active lanes}}{\text{warp lanes}}. \quad (20)$$

If only 16 lanes are active:

$$\eta_{\text{branch}} = \frac{16}{32} = 0.5. \quad (21)$$

Divergence can also occur with loops:

```
for (int k = 0; k < limit[threadIdx.x]; ++k) {  
    acc += values[k];  
}
```

If different threads have different loop bounds, some lanes finish earlier and become inactive while other lanes continue.

3.9 Independent Thread Scheduling

Older NVIDIA GPUs effectively shared a program counter across a warp. Newer architectures, beginning around Volta, introduced more independent thread scheduling.

This allows different lanes of a warp to make progress more flexibly, especially when one group is waiting for memory. However, it also means that programmers must be more careful when relying on implicit warp-synchronous behavior.

When code requires warp-level synchronization, one should use:

```
__syncwarp();
```

This reconverges participating lanes at a known point.

3.10 Conditional Loads and Stores

Not all conditional operations cause harmful branch divergence. Conditional loads and stores are often compiled to predicated instructions, where each lane has a predicate controlling whether it performs the operation.

Example:

```
if (idx < n) {  
    out[idx] = value;  
}
```

This kind of boundary check is common and often necessary. It may be compiled efficiently as a predicated store.

The key distinction is:

- simple conditional memory operations may be predicated,
- large divergent branches with different instruction sequences are more expensive.

4 Hardware-Level Explanation

4.1 How an SM Executes Warps

A simplified SM execution model is:

1. blocks are assigned to the SM,
2. each block is decomposed into warps,
3. warp schedulers choose ready warps,
4. instructions are dispatched to arithmetic units, load-store units, or tensor cores,
5. if a warp stalls, another warp can issue.

The SM does not usually perform an expensive CPU-style context switch between threads. Instead, the register state of many warps is already resident in the SM register file. Switching from one warp to another can therefore be very cheap.

This is a major reason GPUs need many resident warps.

4.2 Latency Hiding

Global memory latency is high. A warp that issues a global load may wait many cycles before the data is available. The GPU hides this latency by scheduling another warp.

Let:

- L_{mem} be memory latency,
- W_{ready} be the number of ready warps,
- I_{warp} be useful independent instructions per warp.

A simplified condition for latency hiding is:

$$W_{\text{ready}} \cdot I_{\text{warp}} \gtrsim L_{\text{mem}}. \quad (22)$$

This is not a precise hardware formula, but it captures the intuition:

The GPU can tolerate long memory latency only if enough other warps are ready to execute while one warp waits.

4.3 SM Resource Constraints

The number of active blocks and warps on an SM is limited by several constraints:

$$B_{\text{resident}} \leq \min \left(B_{\text{max,SM}}, \right. \quad (23)$$

$$\left. \left\lfloor \frac{T_{\text{max,SM}}}{T_{\text{block}}} \right\rfloor, \right. \quad (24)$$

$$\left. \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}} T_{\text{block}}} \right\rfloor, \right. \quad (25)$$

$$\left. \left\lfloor \frac{M_{\text{shared,SM}}}{M_{\text{shared,block}}} \right\rfloor \right). \quad (26)$$

Where:

- $B_{\text{max,SM}}$ is the architectural block limit per SM,
- $T_{\text{max,SM}}$ is the maximum resident threads per SM,
- T_{block} is threads per block,
- R_{SM} is registers per SM,
- R_{thread} is registers per thread,
- $M_{\text{shared,SM}}$ is shared memory available per SM,
- $M_{\text{shared,block}}$ is shared memory used per block.

This formula explains why there is no universally optimal block size. The best block size depends on register use, shared-memory use, instruction mix, memory access pattern, and kernel shape.

4.4 Thread Index Linearization

CUDA allows one-dimensional, two-dimensional, or three-dimensional block layouts. However, warps are formed from a linear ordering of threads.

For a block with dimensions:

$$(\text{blockDim.x}, \text{blockDim.y}, \text{blockDim.z}), \quad (27)$$

the usual linear thread index is:

$$t_{\text{linear}} = \text{threadIdx.x} + \text{blockDim.x}(\text{threadIdx.y} + \text{blockDim.y} \cdot \text{threadIdx.z}). \quad (28)$$

The x dimension is the fastest-moving dimension. Consecutive `threadIdx.x` values are adjacent in the linear order. This matters because consecutive lanes in a warp should ideally access consecutive memory addresses.

4.5 Why the Fastest-Moving Dimension Matters

Suppose a row-major array is indexed as:

$$A[i, j] = A_{\text{base}}[i \cdot N + j]. \quad (29)$$

The column index j is contiguous in memory. If consecutive threads differ in j , memory accesses are contiguous:

$$\text{address}_{\ell} = \text{base} + (i \cdot N + j + \ell) \cdot \text{sizeof}(T), \quad (30)$$

where ℓ is the lane index.

This is good for memory coalescing. If consecutive threads instead differ in i , the stride is N , and memory accesses are scattered:

$$\text{address}_{\ell} = \text{base} + ((i + \ell) \cdot N + j) \cdot \text{sizeof}(T). \quad (31)$$

The lecture defers detailed coalescing to the next session, but the indexing setup is already important here.

4.6 Visual Concept

Draw a GPU as many SM boxes connected to a shared L2 cache and global memory. Inside one SM, draw four scheduler regions, each with a warp scheduler, register file, FP32 units, INT32 units, tensor cores, shared memory, and L1 cache. Then draw thread blocks being assigned to SMs, and each block being split into warps of 32 threads.

5 Mathematical Reasoning

5.1 Kernel Runtime Decomposition

A PyTorch or CUDA workload can spend time in several places:

$$T_{\text{total}} = T_{\text{Python}} + T_{\text{admin}} + T_{\text{data}} + T_{\text{GPU}}. \quad (32)$$

The GPU component can be decomposed as:

$$T_{\text{GPU}} = T_{\text{launch}} + T_{\text{memory}} + T_{\text{compute}}. \quad (33)$$

Where:

- T_{launch} is fixed kernel launch overhead,

- T_{memory} is time spent moving data,
- T_{compute} is time spent executing arithmetic instructions.

The lecture emphasizes that if GPU utilization is low, the first thing to check in real PyTorch programs is often data acquisition. The GPU cannot be fast if it is waiting for the CPU, data loader, disk, network, or preprocessing pipeline.

5.2 Kernel Launch Overhead

A minimal empty CUDA kernel can be used to estimate launch overhead:

```
__global__ void empty_kernel() {
    return;
}
```

If the measured time is approximately:

$$T_{\text{launch}} \approx 3 \mu\text{s}, \quad (34)$$

then any kernel whose useful work takes only a few microseconds is strongly affected by launch overhead.

This motivates kernel fusion. Instead of launching many tiny kernels, combine operations into one larger kernel.

5.3 Memory Bandwidth Time

If a kernel moves B bytes through global memory and the hardware peak bandwidth is β , then the lower bound on memory time is:

$$T_{\text{memory}} \geq \frac{B}{\beta}. \quad (35)$$

For example, if:

$$B = 16 \text{ MB}, \quad (36)$$

$$\beta = 900 \text{ GB/s}, \quad (37)$$

then:

$$T_{\text{memory}} \geq \frac{16 \times 10^6}{900 \times 10^9} \text{ s} \quad (38)$$

$$= 17.78 \times 10^{-6} \text{ s} \quad (39)$$

$$\approx 18 \mu\text{s}. \quad (40)$$

This is a speed-of-light estimate: the kernel cannot move that amount of data faster than the memory subsystem permits.

5.4 Compute Time Lower Bound

If a kernel performs F floating-point operations and the GPU peak FP32 throughput is π_{FP32} , then:

$$T_{\text{compute}} \geq \frac{F}{\pi_{\text{FP32}}}. \quad (41)$$

If it performs I integer operations and the INT32 throughput is π_{INT32} , then:

$$T_{\text{int}} \geq \frac{I}{\pi_{\text{INT32}}}. \quad (42)$$

The relevant compute time is often the maximum among bottlenecked instruction classes:

$$T_{\text{compute}} \approx \max \left(\frac{F}{\pi_{\text{FP32}}}, \frac{I}{\pi_{\text{INT32}}}, T_{\text{special}} \right). \quad (43)$$

Here T_{special} represents special-function operations such as transcendental functions, conversions, or other instructions that may not run at peak FP32 rate.

5.5 RGB to Grayscale Arithmetic Intensity

The lecture discusses an RGB-to-grayscale kernel as a small performance modeling example.

For each pixel, the kernel approximately:

- loads three bytes: red, green, and blue,
- computes one output grayscale value,
- stores one byte,
- performs a few FP32 operations,
- performs some integer indexing operations.

The memory traffic per pixel is approximately:

$$B_{\text{pixel}} = 3 \text{ bytes} + 1 \text{ byte} = 4 \text{ bytes}. \quad (44)$$

A typical grayscale formula is:

$$Y = 0.299R + 0.587G + 0.114B. \quad (45)$$

This involves approximately:

- three multiplications,
- two additions.

Thus:

$$F_{\text{pixel}} \approx 5 \text{ FLOPs}. \quad (46)$$

The arithmetic intensity is:

$$I = \frac{F_{\text{pixel}}}{B_{\text{pixel}}} = \frac{5}{4} = 1.25 \text{ FLOPs/byte}. \quad (47)$$

This is low. Low arithmetic intensity usually means the kernel is memory-bound.

5.6 Roofline Model

The roofline model predicts attainable performance from arithmetic intensity.

Let:

- I be arithmetic intensity in FLOPs per byte,
- β be memory bandwidth in bytes per second,
- π be peak compute throughput in FLOPs per second.

The attainable throughput is bounded by:

$$P(I) = \min(\pi, \beta I). \quad (48)$$

The memory-bound regime occurs when:

$$\beta I < \pi. \quad (49)$$

The compute-bound regime occurs when:

$$\beta I \geq \pi. \quad (50)$$

The transition point is:

$$I^* = \frac{\pi}{\beta}. \quad (51)$$

For a GPU with:

$$\pi = 35.6 \text{ TFLOP/s}, \quad (52)$$

$$\beta = 900 \text{ GB/s}, \quad (53)$$

the ridge point is:

$$I^* = \frac{35.6 \times 10^{12}}{900 \times 10^9} \quad (54)$$

$$= 39.56 \text{ FLOPs/byte}. \quad (55)$$

An RGB-to-grayscale arithmetic intensity of 1.25 FLOPs per byte is far below this ridge point, so it is expected to be memory-bound.

5.7 Why the Roofline Uses a Minimum

Runtime can be approximated by:

$$T \approx \max(T_{\text{memory}}, T_{\text{compute}}), \quad (56)$$

not by the sum, because GPUs can overlap memory waiting with computation from other warps. Throughput is work divided by time, so:

$$P = \frac{F}{T} \approx \frac{F}{\max\left(\frac{B}{\beta}, \frac{F}{\pi}\right)}. \quad (57)$$

This simplifies to:

$$P \approx \min\left(\frac{F}{B}\beta, \pi\right) = \min(I\beta, \pi). \quad (58)$$

This is the roofline formula.

5.8 Matrix Multiplication Data Reuse

For square matrix multiplication:

$$C = AB, \quad (59)$$

where:

$$A \in \mathbb{R}^{N \times N}, \quad B \in \mathbb{R}^{N \times N}, \quad C \in \mathbb{R}^{N \times N}. \quad (60)$$

Each output element is:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}. \quad (61)$$

A naive kernel assigns one thread per output element. Each output element reads:

$$N \text{ values from } A + N \text{ values from } B = 2N \text{ values}. \quad (62)$$

Across all N^2 output elements, the naive global-memory read count is:

$$R_{\text{naive}} = 2N^3 \text{ values}. \quad (63)$$

But each input matrix has only N^2 values, and each value is reused many times. Tiling exploits this reuse by loading blocks of A and B into shared memory.

5.9 Tiled Matrix Multiplication

Let tile size be T . A matrix can be partitioned into blocks:

$$A = \begin{bmatrix} A_{00} & A_{01} & \cdots \\ A_{10} & A_{11} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}, \quad B = \begin{bmatrix} B_{00} & B_{01} & \cdots \\ B_{10} & B_{11} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}. \quad (64)$$

Each output tile is:

$$C_{ij} = \sum_k A_{ik} B_{kj}, \quad (65)$$

where now A_{ik} , B_{kj} , and C_{ij} are $T \times T$ tiles.

For one output tile, the kernel loads:

$$2T^2 \tag{66}$$

values per tile step, and computes:

$$T^3 \tag{67}$$

multiply-add style work per tile step.

The arithmetic intensity of the inner tiled operation improves with T because each loaded value is reused by multiple threads.

A rough tile-level arithmetic intensity is:

$$I_{\text{tile}} \approx \frac{2T^3 \text{ FLOPs}}{2T^2 \cdot 4 \text{ bytes}} = \frac{T}{4} \text{ FLOPs/byte}, \tag{68}$$

assuming FP32 values and counting one multiply-add as two FLOPs.

Thus, increasing tile size improves arithmetic intensity:

$$I_{\text{tile}} \propto T. \tag{69}$$

The limit is that larger tiles require more shared memory and often more registers.

6 Code Walkthrough

6.1 Querying Device Properties in PyTorch

The lecture shows that many hardware constants can be queried rather than memorized.

```
import torch

device = torch.device("cuda")
props = torch.cuda.get_device_properties(device)

print(props.name)
print(props.major, props.minor)
print(props.multi_processor_count)
print(props.regs_per_multiprocessor)
print(props.max_threads_per_multi_processor)
print(props.shared_memory_per_multiprocessor)
```

Line-by-line explanation:

- `torch.device("cuda")` selects the CUDA device.
- `get_device_properties` returns a property object describing the GPU.
- `multi_processor_count` gives the number of SMs.
- `regs_per_multiprocessor` gives the register-file capacity per SM.
- `max_threads_per_multi_processor` gives the resident thread limit per SM.
- `shared_memory_per_multiprocessor` gives the shared-memory capacity per SM.

This information is essential when reasoning about occupancy. A kernel's launch configuration should be compatible with the actual GPU's thread, register, and shared-memory limits.

6.2 Approximate GELU in PyTorch

The lecture uses approximate GELU to show why many small pointwise operations can be slow if each becomes a separate kernel.

The approximate GELU formula is:

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right). \quad (70)$$

A direct PyTorch expression may look like this:

```
import torch
import math

def approximate_gelu_python(x):
    return 0.5 * x * (
        1.0 + torch.tanh(
            math.sqrt(2.0 / math.pi) *
            (x + 0.044715 * torch.pow(x, 3))
        )
    )
```

This expression is mathematically compact, but operationally it may involve many intermediate tensors:

- compute x^3 and store it,
- multiply by 0.044715 and store it,
- add x and store it,
- multiply by $\sqrt{2/\pi}$ and store it,
- apply $\tanh$ and store it,
- add 1 and store it,
- multiply by $0.5x$ and store the final result.

The expensive part is not just arithmetic. The problem is repeated global-memory traffic and repeated kernel launches.

6.3 Fused Approximate GELU CUDA Kernel

A fused CUDA kernel computes the whole formula in one pass:

```
__global__ void gelu_kernel(
    const float* x,
    float* out,
    int n
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n) {
        float v = x[idx];
```

```

    float inner = 0.7978845608f *
        (v + 0.044715f * v * v * v);
    out[idx] = 0.5f * v * (1.0f + tanhf(inner));
}
}

```

Important details:

- `idx` maps one CUDA thread to one tensor element.
- `if (idx < n)` safely handles tensors whose length is not divisible by the block size.
- `v` is loaded once from global memory.
- intermediate values stay in registers.
- the final output is written once to global memory.
- constants use the `f` suffix to avoid accidental double precision.

The memory traffic is approximately:

$$B_{\text{fused}} \approx 4n + 4n = 8n \text{ bytes}, \quad (71)$$

one FP32 read and one FP32 write.

The unfused version may write and read multiple intermediate tensors, giving traffic closer to:

$$B_{\text{unfused}} \approx O(kn), \quad (72)$$

where k is the number of intermediate tensor reads and writes.

Kernel fusion is powerful because it keeps intermediate values in registers instead of materializing them repeatedly in global memory.

6.4 Profiling Many PyTorch Kernels

The lecture uses the PyTorch profiler to show that a Python expression can become many CUDA kernels.

```

import torch
from torch.profiler import profile, ProfilerActivity

x = torch.randn(1_000_000, device="cuda")

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA]
) as prof:
    y = approximate_gelu_python(x)

torch.cuda.synchronize()

print(
    prof.key_averages().table(
        sort_by="cuda_time_total",
        row_limit=20
    )
)

```

```
)  
)
```

What this reveals:

- many pointwise operations may each launch separate kernels,
- each kernel reads inputs and writes outputs,
- launch overhead accumulates,
- intermediate tensors increase global-memory traffic.

This is why PyTorch systems such as nvFuser, TorchInductor, and Triton-based code generation are important. They fuse operations and generate kernels that reduce memory traffic.

6.5 Floating-Point Non-Associativity

The lecture briefly discusses why results may differ up to numerical accuracy.

Floating-point addition is not associative:

$$(a + b) + c \neq a + (b + c) \quad (73)$$

in general.

Example:

```
a = 1.0  
b = 1.0e-8  
c = -1.0  
  
x = (a + c) + b  
y = a + (c + b)  
  
print(x)  
print(y)
```

The mathematical real-number result may be the same, but finite-precision floating point rounds intermediate results. Different kernel fusion choices, different reduction orders, and different compiler transformations can produce small numerical differences.

For FP32, a typical relative accuracy scale is around:

$$\epsilon_{\text{FP32}} \approx 10^{-7}. \quad (74)$$

For FP64, it is around:

$$\epsilon_{\text{FP64}} \approx 10^{-16}. \quad (75)$$

This is why GPU tests often use tolerance-based checks:

```
torch.allclose(y_reference, y_kernel, rtol=1e-5, atol=1e-6)
```

6.6 RGB to Grayscale Kernel

A representative RGB-to-grayscale kernel is:

```
__global__ void rgb_to_gray_kernel(  
    const unsigned char* rgb,  
    unsigned char* gray,  
    int width,  
    int height  
) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    int num_pixels = width * height;  
  
    if (idx < num_pixels) {  
        int base = 3 * idx;  
  
        unsigned char r = rgb[base + 0];  
        unsigned char g = rgb[base + 1];  
        unsigned char b = rgb[base + 2];  
  
        float y = 0.299f * static_cast<float>(r)  
            + 0.587f * static_cast<float>(g)  
            + 0.114f * static_cast<float>(b);  
  
        gray[idx] = static_cast<unsigned char>(y);  
    }  
}
```

Hardware behavior:

- each thread processes one pixel,
- each thread loads three bytes and stores one byte,
- arithmetic intensity is low,
- the kernel is likely memory-bound,
- consecutive threads read consecutive RGB triples,
- consecutive threads write consecutive grayscale bytes.

The theoretical memory lower bound for a 2048×2048 image is:

$$N_{\text{pixels}} = 2048^2 = 4194304, \quad (76)$$

$$B = 4N_{\text{pixels}} \approx 16.78 \text{ MB}. \quad (77)$$

With bandwidth $\beta = 900 \text{ GB/s}$:

$$T_{\text{memory}} \geq \frac{16.78 \times 10^6}{900 \times 10^9} \approx 18.6 \mu\text{s}. \quad (78)$$

Adding launch overhead of around $3 \mu\text{s}$ gives a rough lower bound near:

$$T_{\text{lower}} \approx 21.6 \mu\text{s}. \quad (79)$$

If a measured kernel takes about 26 to 27 μs , then it is relatively close to the simple speed-of-light model.

6.7 Naive Matrix Multiplication Kernel

A simple CUDA matrix multiplication kernel assigns one thread to one output element.

```
__global__ void matmul_naive(
    const float* A,
    const float* B,
    float* C,
    int N
) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        float acc = 0.0f;

        for (int k = 0; k < N; ++k) {
            acc += A[row * N + k] * B[k * N + col];
        }

        C[row * N + col] = acc;
    }
}
```

Line-by-line explanation:

- `row` selects the output row.
- `col` selects the output column.
- each thread computes one C_{ij} .
- the loop over `k` computes the dot product of row i of A and column j of B .
- `A[row * N + k]` accesses consecutive elements as `k` increases.
- `B[k * N + col]` accesses memory with stride N for a single thread.
- across threads with consecutive `col`, accesses to B at fixed `k` are contiguous.

The major inefficiency is repeated global-memory loading. Many threads need the same A and B values, but the naive implementation reloads them from global memory.

6.8 Tiled Matrix Multiplication Kernel

A tiled matrix multiplication kernel uses shared memory to reduce global-memory traffic.

```

#define TILE 16

__global__ void matmul_tiled(
    const float* A,
    const float* B,
    float* C,
    int N
) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int local_row = threadIdx.y;
    int local_col = threadIdx.x;

    int row = blockIdx.y * TILE + local_row;
    int col = blockIdx.x * TILE + local_col;

    float acc = 0.0f;

    for (int tile = 0; tile < N / TILE; ++tile) {
        As[local_row][local_col] =
            A[row * N + tile * TILE + local_col];

        Bs[local_row][local_col] =
            B[(tile * TILE + local_row) * N + col];

        __syncthreads();

        for (int k = 0; k < TILE; ++k) {
            acc += As[local_row][k] * Bs[k][local_col];
        }

        __syncthreads();
    }

    C[row * N + col] = acc;
}

```

This version assumes N is divisible by `TILE`.

Hardware behavior:

- each block computes one 16×16 tile of C .
- the block has $16 \times 16 = 256$ threads.
- each thread loads one element of A into As .
- each thread loads one element of B into Bs .
- `__syncthreads()` ensures the entire tile is loaded before computation starts.
- each thread reuses shared-memory values across the inner loop.
- the second `__syncthreads()` prevents some threads from overwriting shared memory for the next tile while other threads are still using the current tile.

The first synchronization is required because every thread depends on values loaded by other threads. Without it, a thread could read uninitialized or stale shared-memory values.

The second synchronization is required because shared memory is reused on the next tile iteration.

6.9 Tiled Matrix Multiplication with Boundary Checks

For general matrix sizes, N may not be divisible by `TILE`. The kernel should load zeros for out-of-bounds entries.

```
#define TILE 16

__global__ void matmul_tiled_padded(
    const float* A,
    const float* B,
    float* C,
    int N
) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int local_row = threadIdx.y;
    int local_col = threadIdx.x;

    int row = blockIdx.y * TILE + local_row;
    int col = blockIdx.x * TILE + local_col;

    float acc = 0.0f;

    int num_tiles = (N + TILE - 1) / TILE;

    for (int tile = 0; tile < num_tiles; ++tile) {
        int a_col = tile * TILE + local_col;
        int b_row = tile * TILE + local_row;

        if (row < N && a_col < N) {
            As[local_row][local_col] = A[row * N + a_col];
        } else {
            As[local_row][local_col] = 0.0f;
        }

        if (b_row < N && col < N) {
            Bs[local_row][local_col] = B[b_row * N + col];
        } else {
            Bs[local_row][local_col] = 0.0f;
        }

        __syncthreads();

        for (int k = 0; k < TILE; ++k) {
            acc += As[local_row][k] * Bs[k][local_col];
        }

        __syncthreads();
    }
}
```

```

    }

    if (row < N && col < N) {
        C[row * N + col] = acc;
    }
}

```

Important correctness detail:

Do not return early from only some threads in a block before `__syncthreads()`. All threads in the block must reach the synchronization point, otherwise the block can deadlock.

This is why out-of-bounds values are replaced with zeros instead of causing those threads to exit.

6.10 Launching the Tiled Kernel

A typical launch configuration is:

```

dim3 block(TILE, TILE);

dim3 grid(
    (N + TILE - 1) / TILE,
    (N + TILE - 1) / TILE
);

matmul_tiled_padded<<<grid, block>>>(A, B, C, N);

```

Here:

- each block has $TILE^2$ threads,
- each block computes one tile of C ,
- the grid covers all output tiles,
- boundary checks handle partial tiles.

For $TILE = 16$, each block has:

$$T_{\text{block}} = 16 \cdot 16 = 256 \quad (80)$$

threads, a common and reasonable block size.

6.11 Why Tiling Helps but Is Not the End

The lecture notes that tiled matrix multiplication improved performance from roughly 0.9 ms to roughly 0.7 ms in the demonstrated notebook. This is a meaningful speedup, but still far from an optimized GEMM.

The simple tiled kernel does not yet use:

- larger per-thread work,

- register tiling,
- vectorized memory loads,
- double buffering,
- tensor cores,
- warp-level matrix multiply instructions,
- carefully optimized shared-memory layouts,
- asynchronous copies,
- CUTLASS-style hierarchical tiling.

It is still valuable pedagogically because it shows the first major idea:

Load a tile from global memory once, reuse it many times from shared memory, then write the output once.

7 Performance Intuition

7.1 Why Many Blocks Matter

If a GPU has many SMs, the kernel needs enough blocks to occupy them. If the number of blocks is smaller than the number of SMs, then some SMs cannot receive work.

Let:

- S be the number of SMs,
- B be the number of blocks.

A basic occupancy condition is:

$$B \geq S. \tag{81}$$

A stronger practical condition is:

$$B \gg S, \tag{82}$$

because blocks may finish at different times and the scheduler needs enough work to balance across SMs.

7.2 Why Block Size Matters

A block size should usually be a multiple of warp size:

$$T_{\text{block}} = 32m, \tag{83}$$

where m is an integer.

This avoids partially filled warps. Common choices include:

$$128, \quad 256, \quad 512. \quad (84)$$

A block size of 256 gives:

$$\frac{256}{32} = 8 \quad (85)$$

warps per block.

If an SM can hold 1536 threads, then thread count alone allows:

$$\frac{1536}{256} = 6 \quad (86)$$

blocks per SM.

That corresponds to:

$$6 \cdot 8 = 48 \quad (87)$$

resident warps per SM, before other resource limits are considered.

7.3 Why Warp Divergence Wastes Hardware

A warp has 32 lanes. If all lanes execute the same instruction, utilization is high. If only a lanes are active, utilization is:

$$u = \frac{a}{32}. \quad (88)$$

For example:

$$a = 32 \quad u = 1.0, \quad (89)$$

$$a = 16 \quad u = 0.5, \quad (90)$$

$$a = 8 \quad u = 0.25. \quad (91)$$

This is why data-dependent branches inside a warp can be expensive. The hardware may still issue instructions, but many lanes are inactive.

7.4 Why Kernel Fusion Helps

Consider a chain of pointwise operations:

$$y_1 = f_1(x), \quad (92)$$

$$y_2 = f_2(y_1), \quad (93)$$

$$y_3 = f_3(y_2), \quad (94)$$

$$y_4 = f_4(y_3). \quad (95)$$

Without fusion, each stage may read from and write to global memory:

$$x \rightarrow y_1 \rightarrow y_2 \rightarrow y_3 \rightarrow y_4. \quad (96)$$

With fusion, intermediate values remain in registers:

$$x \rightarrow \text{register intermediates} \rightarrow y_4. \quad (97)$$

For a pointwise chain with k intermediate stages, fusion can reduce memory traffic from approximately:

$$O(kN) \quad (98)$$

global operations to:

$$O(N) \quad (99)$$

global operations.

This is the principle behind many PyTorch compiler optimizations.

7.5 Why FlashAttention Is Related

The lecture briefly connects memory locality to FlashAttention. Standard attention materializes the attention matrix:

$$S = QK^T. \quad (100)$$

For sequence length N , this matrix has:

$$N^2 \quad (101)$$

entries. Writing and reading it from high-bandwidth memory is expensive.

FlashAttention avoids materializing the full attention matrix in global memory. It processes blocks of Q , K , and V , using SRAM-like on-chip memory and online softmax statistics.

The high-level principle is the same as tiled matrix multiplication:

Move blocks of data into fast on-chip memory, perform as much useful computation as possible, and avoid unnecessary global-memory reads and writes.

7.6 Memory-Bound Versus Compute-Bound Kernels

A kernel is memory-bound when:

$$T_{\text{memory}} > T_{\text{compute}}. \quad (102)$$

A kernel is compute-bound when:

$$T_{\text{compute}} > T_{\text{memory}}. \quad (103)$$

Using the roofline model, a kernel is memory-bound when:

$$I < I^* = \frac{\pi}{\beta}. \quad (104)$$

It is compute-bound when:

$$I \geq I^*. \quad (105)$$

Low arithmetic intensity operations include:

- vector addition,
- RGB to grayscale,
- simple pointwise activations,
- memory copies,
- elementwise scaling.

High arithmetic intensity operations include:

- matrix multiplication,
- convolution with reuse,
- attention blocks when tiled well,
- tensor-core GEMM.

7.7 Why Shared Memory Helps Matrix Multiplication

In naive matrix multiplication, each value of A and B is loaded repeatedly. In tiled matrix multiplication, a tile of A and a tile of B are loaded once per block and reused by all threads in that block.

If tile size is T , then each loaded value can be reused approximately T times.

The reduction in global-memory reads is roughly proportional to:

$$T. \tag{106}$$

For $T = 16$, the lecture describes reducing repeated reads by a factor around 16.

7.8 Why Synchronization Is Necessary

Shared memory is visible to all threads in a block, but threads execute independently. A thread may reach the compute loop before another thread has finished loading its element into shared memory.

The synchronization:

```
__syncthreads();
```

acts as a block-level barrier:

$$\text{all threads arrive} \implies \text{all threads may proceed.} \tag{107}$$

This ensures shared-memory data is complete before it is consumed.

The second synchronization in tiled matrix multiplication prevents overwrite hazards:

$$\text{finish using tile } k \implies \text{safe to load tile } k + 1. \tag{108}$$

8 Common Misconceptions

8.1 Misconception: A CUDA Thread Is Like a CPU Thread

A CUDA thread is much lighter than a CPU thread. CUDA threads are grouped into warps and scheduled in bulk. The GPU relies on many resident threads to hide latency.

A CPU thread is a much heavier software and operating-system abstraction. A CUDA thread is closer to a lane of a massive throughput machine.

8.2 Misconception: More Threads Always Means More Speed

More threads help only if they increase useful parallelism and latency hiding. Too many threads per block can reduce occupancy if they consume too many registers or shared memory.

The real objective is not maximum thread count. The objective is enough active warps, efficient memory access, low divergence, and high useful work per byte.

8.3 Misconception: High Occupancy Guarantees High Performance

High occupancy is helpful, but it does not guarantee speed. A kernel with high occupancy can still be slow due to:

- low arithmetic intensity,
- uncoalesced memory accesses,
- excessive global-memory traffic,
- synchronization overhead,
- inefficient instruction mix,
- poor data reuse.

8.4 Misconception: Shared Memory Automatically Makes Code Fast

Shared memory helps only when it enables reuse or improves access patterns. If data is loaded into shared memory and used only once, the kernel may become slower because of extra instructions and synchronization.

Shared memory is useful when:

$$\text{reuse benefit} > \text{load, store, and synchronization cost.} \quad (109)$$

8.5 Misconception: Branches Are Always Bad

Branches are not always bad. Branches are expensive mainly when threads in the same warp take different paths.

If all lanes in a warp take the same branch, there is little or no divergence penalty.

Boundary checks such as:

```
if (idx < n) {  
    out[idx] = value;  
}
```

are often necessary and can be compiled efficiently.

8.6 Misconception: FP64 Is Just More Accurate FP32

On consumer GPUs, FP64 may be dramatically slower than FP32. Accidentally using double-precision constants or 64-bit arithmetic can reduce performance.

Use `0.5f` instead of `0.5` when writing FP32 CUDA kernels.

8.7 Misconception: Local Memory Is Fast Because It Sounds Local

CUDA local memory is local to a thread in the programming model, but it is often physically stored in global memory. It is much slower than registers. Register spills to local memory can badly hurt performance.

8.8 Misconception: Blocks Can Coordinate Globally

Normal CUDA thread blocks cannot safely synchronize with each other during a kernel. Blocks must be independent unless using special cooperative-group mechanisms. The scheduler may run blocks in any order.

9 Practical Takeaways

9.1 Think in Hardware Resources

When writing a CUDA kernel, always ask:

- How many blocks are launched?
- How many threads are in each block?
- How many warps are in each block?
- How many registers does each thread use?
- How much shared memory does each block use?
- How many blocks can fit on one SM?
- Are enough warps resident to hide latency?

9.2 Use Reasonable Block Sizes

Start with block sizes such as:

$$128, \quad 256, \quad 512. \tag{110}$$

For two-dimensional tiling, $16 \times 16 = 256$ threads is a common educational starting point.

Avoid choosing a block size only because it is large. Large blocks can reduce scheduling flexibility.

9.3 Avoid Accidental Slow Types

Use FP32 constants in FP32 kernels:

```
float a = 0.044715f;
float b = 0.7978845608f;
```

Prefer 32-bit indexing when safe:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

Use 64-bit indexing only when the tensor size requires it.

9.4 Reduce Global-Memory Traffic

The main memory rule is:

Read from global memory as little as possible, reuse values in registers or shared memory, and write final results once.

This principle appears in:

- fused activations,
- tiled matrix multiplication,
- convolution tiling,
- FlashAttention,
- compiler fusion systems such as TorchInductor and nvFuser.

9.5 Use Roofline Reasoning Before Optimizing

Before optimizing, estimate arithmetic intensity:

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}. \quad (111)$$

Then compare with the ridge point:

$$I^* = \frac{\text{peak FLOP/s}}{\text{memory bandwidth}}. \quad (112)$$

If $I \ll I^*$, the kernel is likely memory-bound. Optimizing arithmetic instructions may not help much. Focus on memory traffic, fusion, coalescing, and reuse.

If $I \gg I^*$, the kernel is likely compute-bound. Focus on instruction throughput, tensor cores, register blocking, and math pipeline utilization.

9.6 Check Assumptions Experimentally

The lecture emphasizes that when a book, speaker, or mental model says something about thread ordering or warp behavior, it is useful to write small kernels to test the assumption.

GPU programming has many architecture-dependent details. Good engineers verify behavior with:

- small CUDA experiments,
- PyTorch profiling,
- Nsight Systems,
- Nsight Compute,
- occupancy calculators,
- device property queries.

10 Project or Experiment Ideas

10.1 Experiment 1: Measure Kernel Launch Overhead

Write an empty CUDA kernel and measure its runtime with CUDA events.

Expected result:

$$T_{\text{launch}} \approx \text{a few microseconds.} \quad (113)$$

Then compare launching one fused kernel versus many tiny kernels.

10.2 Experiment 2: PyTorch GELU Versus Custom Fused GELU

Implement approximate GELU three ways:

1. direct Python PyTorch expression,
2. `torch.nn.functional.gelu` with approximate mode,
3. custom fused CUDA kernel.

Measure:

- runtime,
- number of CUDA kernels launched,
- global-memory traffic if available in profiler,
- numerical difference.

10.3 Experiment 3: RGB to Grayscale Speed-of-Light Model

Implement RGB to grayscale. Compute:

$$I = \frac{5}{4} = 1.25 \text{ FLOPs/byte.} \quad (114)$$

Estimate the memory lower bound using GPU bandwidth. Compare measured time against:

$$T_{\text{estimated}} = T_{\text{launch}} + \frac{B}{\beta}. \quad (115)$$

This teaches how to judge whether a simple memory-bound kernel is already close to optimal.

10.4 Experiment 4: Naive Versus Tiled Matrix Multiplication

Implement:

- naive one-thread-per-output matmul,
- tiled matmul with `TILE = 8`,
- tiled matmul with `TILE = 16`,
- tiled matmul with `TILE = 32`, if resources permit.

Measure:

- runtime,
- achieved GFLOP/s,
- shared-memory usage,
- occupancy,
- register count,
- correctness versus `torch.matmul`.

The achieved throughput is:

$$P_{\text{matmul}} = \frac{2N^3}{T_{\text{runtime}}}. \quad (116)$$

10.5 Experiment 5: Register Pressure

Write several versions of a kernel with increasing numbers of local variables. Compile with verbose PTXAS output and inspect register usage.

The useful compiler flag is conceptually:

`-Xptxas=-v`

Measure how increasing registers per thread changes occupancy and runtime.

10.6 Experiment 6: Warp Divergence

Write two kernels:

```
if (threadIdx.x < 16) {
    out[idx] = a[idx] + b[idx];
} else {
    out[idx] = a[idx] * b[idx];
}
```

and:

```
if (blockIdx.x % 2 == 0) {
    out[idx] = a[idx] + b[idx];
} else {
    out[idx] = a[idx] * b[idx];
}
```

The first branch diverges within a warp. The second may branch uniformly across many warps. Compare performance.

10.7 Experiment 7: Device Property Explorer

Write a small script that prints:

- SM count,
- max threads per SM,
- max threads per block,
- registers per SM,
- shared memory per SM,
- warp size,
- memory size.

Then compute theoretical occupancy for several block sizes.

11 Final Mental Model

A CUDA kernel is not fast just because it uses many threads. It is fast when the launched grid creates enough independent blocks to occupy all SMs, each SM has enough resident warps to hide latency, each warp executes with minimal divergence, and the kernel performs substantial useful work for every byte moved from global memory.

The GPU execution model can be compressed into this mental model:

Blocks are scheduled onto SMs. Blocks split into warps. Warps issue instructions when ready. Registers hold per-thread state. Shared memory enables cooperation within a block. Global memory is large but expensive. Performance comes from keeping SMs busy while moving as little data as possible through global memory.

The roofline model gives the performance compass:

$$P(I) = \min(\pi, \beta I). \quad (117)$$

If arithmetic intensity I is low, optimize memory movement. Fuse kernels, coalesce accesses, and reuse data in shared memory or registers. If arithmetic intensity is high, optimize compute throughput using better tiling, more per-thread reuse, tensor cores, and instruction-level efficiency.

The lecture's practical GPU programming lesson is:

To write fast CUDA, do not only count FLOPs. Count bytes, count warps, count registers, count shared memory, and count how many times each value is reused before returning to global memory.